

4 Événements React

4.1 Syntaxe des Événements

React utilise une syntaxe similaire au HTML mais avec **camelCase**.

Listing 8 – Événements courants

```
// onClick - Clic sur élément
<button onClick={handleClick}>Cliquer</button>

// onChange - Changement input
<input onChange={handleChange} />

// onSubmit - Soumission formulaire
<form onSubmit={handleSubmit}>...</form>

// onMouseEnter - Survol souris
<div onMouseEnter={handleHover}>Survolez-moi</div>

// onKeyDown - Appui touche clavier
<input onKeyDown={handleKeyPress} />
```

△ Important

Différences HTML vs React :

```
// HTML (minuscules)
<button onclick="handleClick()">Click</button>

// React (camelCase + fonction)
<button onClick={handleClick}>Click</button>
```

4.2 Gestionnaires d'Événements (Event Handlers)

Méthode 1 : Fonction nommée

Listing 9 – Handler nommé

```
function Button() {
  const [count, setCount] = useState(0);

  const handleClick = () => {
    setCount(count + 1);
    console.log('Bouton cliqué !');
  };

  return <button onClick={handleClick}>Cliquer</button>;
}
```

Méthode 2 : Fonction inline

Listing 10 – Handler inline

```
function Button() {
  const [count, setCount] = useState(0);

  return (
    <button onClick={() => setCount(count + 1)}>
```

```
        Cliquer  
    </button>  
    );  
}
```

★ Astuce

Bonnes pratiques :

- Logique simple (1 ligne) → Inline
- Logique complexe → Fonction nommée
- Nommer handlers avec "handle" : `handleClick`, `handleSubmit`

4.3 L'Objet Event

Les handlers reçoivent automatiquement un objet `event`.

Listing 11 – Utilisation objet event

```
function Form() {
  const handleSubmit = (event) => {
    event.preventDefault(); // Empêcher rechargement page
    console.log('Formulaire soumis');
  };

  const handleInput = (event) => {
    console.log(event.target.value); // Valeur input
  };

  return (
    <form onSubmit={handleSubmit}>
      <input onChange={handleInput} />
      <button type="submit">Envoyer</button>
    </form>
  );
}
```

Propriétés utiles de event :

- `event.target` : Élément qui a déclenché l'événement
- `event.target.value` : Valeur d'un input
- `event.preventDefault()` : Annuler comportement par défaut
- `event.stopPropagation()` : Arrêter propagation événement

4.4 Passer des Paramètres aux Handlers

Listing 12 – Handlers avec paramètres

```
function ProductList() {
  const handleDelete = (productId) => {
    console.log(`Supprimer produit ${productId}`);
  };

  const handleEdit = (productId, newName) => {
    console.log(`Éditer ${productId} : ${newName}`);
  };

  return (
    <div>
      {/* Méthode 1 : Arrow function inline */}
      <button onClick={() => handleDelete(123)}>
        Supprimer
      </button>

      {/* Méthode 2 : Avec event + paramètres */}
      <button onClick={(e) => handleEdit(123, 'Nouveau nom')}>
        Éditer
      </button>
    </div>
  );
}
```

△ Important

Erreur fréquente :

```
// MAUVAIS - Appelle immédiatement la fonction
<button onClick={handleClick()}>Click</button>

// BON - Passe la fonction (sera appelée au clic)
<button onClick={handleClick}>Click</button>

// BON - Arrow function pour appeler avec paramètres
<button onClick={() => handleClick(123)}>Click</button>
```

PAUSE COURS - Exercice Papier 2 (Events)

Durée : 20 minutes

Exercice : Créer composant `ToggleButton` avec :

- State `isOn` (boolean, initial = false)
- Bouton qui toggle `isOn` au clic
- Affichage texte : "ON" si `isOn=true`, "OFF" sinon
- Background vert si ON, rouge si OFF

Bonus : Ajouter compteur de clics